

ZEQUI

AI Study & Growth Companion

Project Architecture • Core Idea • Workflow • Technology Overview

LinkedIn Project Showcase

Confidential implementation details intentionally omitted.

1. What is ZEQUI?

ZEQUI is a web-based AI companion designed around study, learning, focus and personal growth workflows. The interface brings several AI-assisted activities into one application rather than requiring a separate tool for every task.

Core experience

A user interacts with ZEQUI through the web interface, chooses a capability such as Chat or Summarize, provides input, and receives an AI-generated response. Supporting UI state, local persistence, file handling and optional services sit around that central interaction.

The public-facing concept can be described without exposing private implementation details, API credentials, environment values, internal prompts, or deployment secrets.

2. High-Level Architecture

User → React UI → Feature/State Logic → Secure AI Routing Layer → AI Provider

Layer	Role
Presentation	React + TypeScript interface, responsive screens, controls and reusable components.
Application features	Chat, Summarize, Research, Advice, Companion and related learning tools.
State & hooks	Manages UI state, profiles/preferences, storage, AI requests and supporting behaviors.
Utilities	Input handling, file processing, caching and other reusable application helpers.
Server-side boundary	A serverless routing layer keeps sensitive AI configuration away from the browser.
External services	AI model provider and other application services used by the project.
Browser capabilities	Local storage and browser speech/file capabilities where supported.

Note: This diagram intentionally describes roles and data flow rather than exposing private source-level implementation.

3. How the Application Works

① User input

The user types a question, supplies study material, or selects an application capability.

② UI validation & preparation

The client prepares the request and applies the application's input-handling logic.

③ Feature logic

The selected feature determines the kind of AI task and the relevant application state.

④ Secure request path

AI-related requests can pass through the server-side routing boundary instead of exposing sensitive configuration in the browser.

⑤ AI processing

The configured AI provider processes the request and returns a response.

⑥ Response handling

ZEQUI receives the result and presents it through the appropriate interface.

⑦ Supporting actions

The application can provide actions such as reading, exporting, clearing, quiz generation or persistence depending on the feature.

4. Main Modules

The project is organized around reusable UI and feature components. The exact internal boundaries may evolve as the project grows.

Module	Purpose
Chat	General AI conversation and response display.
Summarize	Turns supplied study material into a concise or deeper summary workflow.
Research	Provides an interface for research-oriented AI interaction.
Advice	Provides guidance-oriented interaction.
Companion	A more personalized assistant experience using the project's profile/persona concepts.
Quiz / learning tools	Supports active recall and learning-focused interactions.
Focus / tracking UI	Surfaces study priorities, topics and related progress information.
Voice / accessibility	Uses supported browser capabilities for speech/read-back interactions.
Export / file handling	Supports moving information into or out of the application where implemented.

5. Technology Stack

Frontend: React, TypeScript and Vite form the main web application foundation. Styling and interface utilities support the dark, modern ZEQUI experience.

Application logic: Custom hooks and reusable utilities separate UI concerns from state, storage, AI requests and supporting functionality.

AI integration: The application uses an AI-provider routing approach. Private credentials/configuration are intentionally not documented in this public showcase.

Browser services: Local storage, file processing and supported browser speech capabilities can provide client-side supporting functionality.

6. Security & Confidentiality

For a public portfolio, the following should stay private: API keys, access tokens, secret environment variables, private endpoints, credentials, authentication secrets, proprietary prompts, internal deployment values and any personal user data.

This showcase therefore focuses on architecture, user-visible behavior and the engineering concept rather than reproducing sensitive implementation details.

7. Example User Flow

Stage	Example
Start	User opens ZEQUI and selects Chat.
Ask	User enters a learning question.
Prepare	The application prepares the request.
Route	The request crosses the protected server-side boundary.
Generate	The AI provider generates a response.
Display	ZEQUI renders the response in the conversation interface.
Learn	The user can continue, generate a quiz, read the response aloud or use another learning feature.

8. Portfolio Summary

ZEQUI demonstrates a practical full-stack AI application pattern: a React/TypeScript client, modular feature architecture, reusable state and utility layers, a server-side boundary for AI requests, and a user-focused learning interface. The public architecture communicates the engineering approach while keeping confidential implementation details private.